Python Functions

Complete Reference Notes

Shwetank's Python Journey • Functions Module

Functions are reusable blocks of code that perform a specific task. They help organise code, avoid repetition, and make programs easier to read and test. This document covers 10 essential function concepts in Python, from basic syntax to advanced topics like generators and recursion.

Quick Reference

#	Concept	Key Syntax / Feature
1	**Basic Function Syntax**	`def func_name(param): → return value`
2	**Multiple Parameters**	`def func(a, b): → multiple inputs`
3	**Polymorphism**	`Works on ints and strings with same operator`
4	**Multiple Return Values**	`return val1, val2 → tuple unpacking`
5	**Default Parameter**	`def func(name='default'):`
6	**Lambda Function**	`lambda x: expression`
7	***args (Variable Args)**	`def func(*args): → accepts any # of args`
8	****kwargs (Keyword Args)**	`def func(**kwargs): → key=value pairs`
9	**Generator / yield**	`def gen(): yield value → lazy iteration`
10	**Recursive Function**	`Function that calls itself`

1. Basic Function Syntax

A function is defined using the def keyword, followed by the function name, parentheses, and a colon. Use return to send a value back to the caller.

```
def square(n):
    return n * n

# Call the function
result = square(5)
print(result)    # Output: 25
```

```
# Works with floats too
print(square(3.5))   # Output: 12.25
```

TIP Function names should be lowercase with underscores (snake_case). Choose descriptive names like square() rather than s() — readability matters!

? Interview Q: What happens if a function has no return statement?

☑ Python implicitly returns None. Always use return explicitly when you need a value back.

2. Function with Multiple Parameters

Functions can accept multiple parameters separated by commas. Python matches arguments to parameters positionally.

```
def add(a, b):
    return a + b

print(add(3, 7))      # Output: 10
print(add(2.5, 4.5))  # Output: 7.0

# You can also use keyword arguments
print(add(b=10, a=5))  # Output: 15
```

NOTE When calling with keyword arguments (add(b=10, a=5)), order does not matter. Keyword args always override positional order.

3. Polymorphism in Functions

Polymorphism means the same function can work on different data types. In Python, the * operator works on both numbers (multiplication) and strings (repetition), so a single multiply() function handles both!

```
def multiply(a, b):
    return a * b

# Works with numbers
print(multiply(4, 5))         # Output: 20

# Works with strings!
print(multiply('chai', 3))    # Output: 'chaichaichai'
```

```
# Works with lists too
print(multiply([1, 2], 3))    # Output: [1, 2, 1, 2, 1, 2]
```

NOTE	This works because Python's * operator is defined (via dunder methods like __mul__) for many built-in types. The function itself doesn't need to know the type — Python figures it out at runtime.

? Interview Q: What is duck typing in Python?

☑ Duck typing means Python doesn't check types — it just tries the operation. 'If it walks like a duck and quacks like a duck, it is a duck.' Polymorphism in Python relies on duck typing.

4. Function Returning Multiple Values

Python functions can return multiple values as a tuple. You can unpack them directly into separate variables on the calling side.

```
import math

def circle_stats(radius):
    area = math.pi * radius ** 2
    circumference = 2 * math.pi * radius
    return area, circumference   # returns a tuple

# Tuple unpacking
a, c = circle_stats(5)
print(f'Area: {a:.2f}')           # Area: 78.54
print(f'Circumference: {c:.2f}')  # Circumference: 31.42

# Or capture the whole tuple
result = circle_stats(5)
print(result)   # (78.539..., 31.415...)
```

TIP	Use f-strings with :.2f to format floats to 2 decimal places — great for math results!

5. Default Parameter Value

Parameters can have default values, making them optional. If the caller doesn't pass an argument for that parameter, the default is used. Parameters with defaults must come after those without.

```
def greet(name='Guest'):
    return f'Hello, {name}! Welcome!'
```

```
print(greet())            # Hello, Guest! Welcome!
print(greet('Shwetank')) # Hello, Shwetank! Welcome!

# Multiple defaults
def make_chai(tea='Darjeeling', milk=True, sugar=1):
    extras = 'with milk' if milk else 'without milk'
    return f'{tea} chai, {extras}, {sugar} spoon sugar'

print(make_chai())                    # uses all defaults
print(make_chai('Assam', sugar=2))    # override some
```

⚠ WARN	Never use mutable objects (like lists or dicts) as default parameter values! They are shared across all calls. Use None as the default and create the object inside the function instead.

? Interview Q: Why shouldn't you use def func(items=[]) as a default?

☑ The list is created once when the function is defined, not each time it's called. All calls share the same list object — causing subtle bugs. Use def func(items=None): items = items or [] instead.

6. Lambda Function

A lambda is a small, anonymous (nameless) function defined in a single line. It can take any number of arguments but only one expression. Lambdas are often used for short operations or passed as arguments to higher-order functions.

```
# Syntax: lambda arguments: expression

# Basic example: cube of a number
cube = lambda x: x ** 3
print(cube(3))    # Output: 27
print(cube(4))    # Output: 64

# Lambda with multiple arguments
add = lambda a, b: a + b
print(add(10, 5))  # Output: 15

# Useful with sorted()
teas = ['Assam', 'darjeeling', 'Green']
sorted_teas = sorted(teas, key=lambda t: t.lower())
print(sorted_teas)  # ['Assam', 'darjeeling', 'Green']
```

NOTE	Lambdas are best for simple, one-off operations. If the logic is complex or reused, write a proper def function instead — it is clearer and easier to debug.

7. Function with *args

*args allows a function to accept any number of positional arguments. Inside the function, args behaves as a tuple containing all the extra arguments passed in.

```
def total(*args):
    print(type(args))   # <class 'tuple'>
    return sum(args)

print(total(1, 2, 3))         # 6
print(total(10, 20, 30, 40))  # 100
print(total(5))               # 5

# Mix regular params and *args
def order_chai(size, *flavours):
    print(f'Size: {size}')
    for f in flavours:
        print(f'  Flavour: {f}')

order_chai('large', 'Masala', 'Adrak', 'Elaichi')
# Size: large
# Flavour: Masala  ...etc
```

TIP	The name args is just a convention — you could write *items or *numbers. The * is what matters. But stick to *args for readability.

? Interview Q: What is the difference between *args and a list parameter?

☑ With a list parameter, the caller must pass an actual list: func([1,2,3]). With *args, the caller passes values directly: func(1,2,3). *args is more ergonomic for variable-length calls.

8. Function with **kwargs

**kwargs lets a function accept any number of keyword arguments. Inside the function, kwargs is a dictionary of key-value pairs. This is ideal when you don't know in advance which named arguments will be passed.

```
def print_info(**kwargs):
    print(type(kwargs))   # <class 'dict'>
```

```
    for key, value in kwargs.items():
        print(f'{key}: {value}')

print_info(name='Shwetank', city='Kolkata', lang='Python')
# name: Shwetank
# city: Kolkata
# lang: Python

# Combining all parameter types
def chai_order(size, *flavours, **extras):
    print(f'Size: {size}, Flavours: {flavours}')
    print('Extras:', extras)

chai_order('small', 'Adrak', 'Masala', milk='oat', sugar='none')
```

NOTE Parameter order rule: def func(regular, *args, **kwargs) — regular params first, then *args, then **kwargs. Python enforces this order.

? Interview Q: When would you use **kwargs in real code?

☑ When building flexible APIs or wrapper functions — e.g. a logging utility that accepts any metadata fields, or a function that wraps another function and passes along keyword arguments with **kwargs.

9. Generator Function with yield

A generator function uses yield instead of return. It produces values lazily — one at a time, only when requested. This is memory-efficient for large sequences since it doesn't build the entire list in memory.

```
def even_numbers(limit):
    n = 0
    while n <= limit:
        yield n      # pause and send value out
        n += 2       # resume from here next time

# Using the generator
for num in even_numbers(10):
    print(num, end=' ')  # 0 2 4 6 8 10

# Generators are also iterators
gen = even_numbers(6)
print(next(gen))   # 0
print(next(gen))   # 2
```

```
print(next(gen))   # 4

# Convert to list if needed
print(list(even_numbers(8)))  # [0, 2, 4, 6, 8]
```

TIP	Generators are perfect when working with large datasets (files, database rows, infinite sequences). They use O(1) memory regardless of how many values they produce.

? Interview Q: What is the difference between return and yield?

☑ return exits the function permanently and sends one value back. yield pauses execution and sends a value back, then resumes from that exact point the next time next() is called. A function with yield becomes a generator object.

10. Recursive Function

A recursive function is one that calls itself to solve a smaller version of the same problem. Every recursive function needs a base case (when to stop) and a recursive case (how to break the problem down).

```
def factorial(n):
    # Base case: stop recursion
    if n == 0 or n == 1:
        return 1
    # Recursive case: call itself
    return n * factorial(n - 1)

print(factorial(5))   # 120  (5×4×3×2×1)
print(factorial(0))   # 1
print(factorial(1))   # 1

# How it unwinds:
# factorial(5)
#   5 * factorial(4)
#       4 * factorial(3)
#           3 * factorial(2)
#               2 * factorial(1)
#                   return 1    ← base case
# = 5 * 4 * 3 * 2 * 1 = 120
```

⚠ WARN	Python has a default recursion limit of 1000 calls. For very deep recursion, you will get a RecursionError. For large inputs, prefer an iterative approach or use sys.setrecursionlimit() carefully.

? Interview Q: What are the two essential parts of a recursive function?

☑ 1) Base case — the condition that stops the recursion (prevents infinite calls). 2) Recursive case — where the function calls itself with a simpler/smaller input, making progress toward the base case.

Summary: Functions Cheat Sheet

Concept	One-liner
`def / return`	Core building blocks. def names it, return sends the result back.
`Multiple params`	Comma-separate params; call by position or by name.
`Polymorphism`	Same function, different types — Python's duck typing at work.
`Multiple returns`	Return a tuple; unpack with a, b = func().
`Default params`	Assign in def; mutable defaults are a common bug trap.
`Lambda`	lambda x: x**3 — one-liner anonymous function.
`*args`	Captures any number of positional args as a tuple.
`**kwargs`	Captures any number of keyword args as a dict.
`yield / generator`	Lazy, memory-efficient value production — use next() or for loops.
`Recursion`	Self-calling function — always needs a base case to stop.

Keep experimenting in the REPL — functions click fastest when you play with them! 🪐